PA-DSE: Feasibility-Aware Design Space Exploration for High-Level Synthesis via Hierarchical Evidence-Bounded Pruning

Anonymous Authors

***Abstract*—High-Level Synthesis (HLS) design-space exploration (DSE) is often bottlenecked by synthesis failures: on our Bambu suite, uniform-random sampling wastes 85% of a 60-call budget on infeasible configurations. Prior work either ignores the failure signal (classical optimizers) or predicts it with offline-trained surrogates (learned classifiers), each with known limitations in interpretability and warm-up cost.**

We present PA-DSE, a feasibility-aware DSE policy built on one principle: *action strength must not exceed evidence strength*. PA-DSE has two layers. Layer 1, the Static Constraint Filter (SCF), permanently removes configurations matching tool-documented rules. Layer 2, the Dynamic Failure Risk Learner (DFRL), learns online during a single run through two sub-components: a Recurrent Pattern Extractor (RPE) that hard-skips configurations matching learned failure signatures, and an Online Failure Risk Scorer (OFRS) that reorders the remaining queue by a smoothed per-dimension risk score.

Across 12 benchmarks on two structurally different HLS tools (PandA-Bambu and Dynamatic), PA-DSE reaches 92.5% success rate on Bambu (vs. 86.0% for an AutoScaleDSE-style [1] random-forest classifier) and 92.0% on Dynamatic (vs. 90.1% for GP-BO [2]), with 2–$11\times$ fewer wasted synthesis calls and negligible overhead ($< 0.22\%$ of synthesis cost). An 8-way ablation confirms the evidence hierarchy: OFRS's weak-evidence reordering is the dominant contributor; SCF is a safety net whose benefit depends on the tool exposing documented rules; allowing OFRS to hard-skip breaks the hierarchy, with an order-of-magnitude variance increase. PA-DSE is training-free, single-run, and *decision-traceable*: every skip is attributable to a named rule or signature. Code and data: https://github.com/jane09250410/hls-dse.

***Index Terms*—High-Level Synthesis, Design Space Exploration, Feasibility-Aware Optimization, Interpretable Machine Learning, PandA-Bambu, Dynamatic.**

I. Introduction

HIGH-Level Synthesis (HLS) enables rapid generation of hardware accelerators from C/C++ specifications, but the resulting design space is large and irregular. A single benchmark kernel may expose dozens of synthesis pragmas, each controlling loop pipelining, memory partitioning, array-to-channel mapping, or resource binding. For two representative HLS tools—PandA-Bambu [3] and Dynamatic [4]—the configurations we consider in this work number 420 and 192 respectively, and the space grows combinatorially with kernel complexity. Exhaustive exploration is impractical: a single synthesis run typically takes 2–10 seconds for small kernels and tens of seconds for larger ones, so brute-force evaluation of even a moderately sized space consumes hours of wall-clock time.

Submitted for blind review. Author information withheld.

Design Space Exploration (DSE) methods attempt to navigate this space efficiently. Classical approaches draw on general-purpose optimization: simulated annealing, genetic algorithms, and Bayesian optimization with Gaussian Processes [5] have all been adapted to HLS DSE [2], [6], [7]. More recent work applies machine learning to *predict* synthesis outcomes before invoking the tool [1], [8]–[11], potentially reducing the number of expensive tool invocations. Both families, however, share a blind spot: they treat synthesis failure as just another low-objective sample, rather than as a signal carrying structural information about which regions of the design space are *systematically* infeasible.

Feasibility is the dominant cost driver. In our experiments on eight Bambu benchmarks, roughly one-third of configurations in the raw design space are provably infeasible due to documented parameter incompatibilities (e.g., MEM_ACC_11 memory controllers with multi-channel binding). A larger fraction—often more than half—are *likely* infeasible due to source-level patterns that interact poorly with pipelining (accumulation loops, control-flow branches inside loops). Random sampling against this space achieves only 15% success rate (SR) at budget B=60 on Bambu. The problem is not that a good sampler cannot find feasible configurations; it is that most of the budget is wasted on configurations the tool will reject, often with error messages that are informative if anyone bothers to read them.

The core idea. PA-DSE is a *feasibility-aware* DSE policy that learns from tool error messages online, during a single exploration run, without offline training or surrogate modeling. The policy is organized around a single design principle we call the *evidence hierarchy*: the strength of the action taken on a configuration must not exceed the strength of the evidence justifying that action.

PA-DSE has two layers, the second of which contains two complementary sub-components:

- **Layer 1 (SCF, Static Constraint Filter).** Configurations matching tool-documented incompatibility rules are permanently removed before exploration begins. This is the *strongest* action (cross-run permanent block), reserved for *proven* constraints with zero false-positive risk. Configurations matching weaker source-level heuristics are *deferred* (placed at the queue tail) but not removed; they may be promoted later by online evidence.
- **Layer 2 (DFRL, Dynamic Failure Risk Learning).** An online policy that accumulates empirical evidence during

a single exploration run. It has two sub-components that operate at different evidence strengths:

- **RPE (Recurrent Pattern Extractor).** After repeated observations of the same error type, RPE extracts typed failure *signatures*—parameter conditions that consistently co-occur with that error type and discriminate failures from successes. Matching configurations are hard-skipped for the remainder of the run, subject to a small probe rate that keeps signatures honest.
- **OFRS (Online Failure Risk Scorer).** A lightweight per-dimension failure-rate tracker that reorders the remaining queue each iteration by a smoothed risk score. OFRS never skips configurations; it only changes their evaluation order. This is the *weakest* action, chosen deliberately because per-dimension marginal statistics are too noisy to justify permanent exclusion.

The two layers are complementary, not redundant, and their interplay is tool-dependent. On Bambu, SCF removes 33% of the space as provably infeasible; disabling it causes DFRL to be contaminated by catastrophic configurations and drops SR from 92% to 70%. On Dynamatic, the tool exposes no documented incompatibility rules ($\mathcal{C}_{\text{blk}} = \emptyset$), so SCF is a no-op; DFRL alone drives the 92% SR, validating that DFRL is not parasitic on SCF. The slogan is: *SCF for tools with static constraints, DFRL for all tools.*

Contributions. This paper makes four contributions.

1) **Policy design.** We propose PA-DSE, a two-layer feasibility-aware DSE policy organized around the *evidence hierarchy* principle. Layer 1 (SCF) handles static, tool-documented constraints; Layer 2 (DFRL) is online and contains two sub-components (RPE and OFRS) that operate at different evidence strengths. Each layer's action strength is explicitly justified by the evidence it has access to (§III).
2) **Empirical validation across tools.** We evaluate PA-DSE on two qualitatively different HLS tools—Bambu (static scheduling, C-centric) and Dynamatic (elastic dataflow, LLVM-based)—across 12 benchmarks (eight Bambu, four Dynamatic). PA-DSE achieves 92.5% SR on Bambu (vs. 86.0% for the best baseline) and 92.0% SR on Dynamatic (vs. 90.1% for the best baseline), consistently across seeds and queue permutations (§V).
3) **Mechanism isolation.** Through an 8-way ablation we show that (a) OFRS reordering is the dominant contributor to within-run performance, (b) SCF is essential as a safety net preventing DFRL from learning from catastrophic configurations, and (c) RPE adds targeted coverage for type-specific failure modes. The evidence hierarchy is empirically grounded (§V-G).
4) **Near-zero runtime overhead.** The algorithmic overhead of PA-DSE is 5.3 ms per iteration on Bambu and 2.2 ms per iteration on Dynamatic, against synthesis times of 2.5 s and 6.2 s respectively—ratios of $1:474$ and $1:2908$ (§V-H).

Honest scope. PA-DSE is not a general-purpose synthesis prediction model. It does not generalize zero-shot across benchmarks (RPE signatures are per-run and do not persist), and its static rules are tool-specific. Its strength is on-the-fly, within-run adaptation with decision-traceable, inspectable state. We discuss these limitations and their implications in §VI.

II. Background and Related Work

A. HLS Design Space and Feasibility

High-Level Synthesis compiles a C/C++ specification into RTL, subject to user-specified pragmas. The pragmas form a combinatorial *configuration space* $\mathcal{C}$: each configuration $c \in \mathcal{C}$ is a discrete point (choice of loop pipelining, memory controllers, channel count, clock period target, etc.). Given a synthesis tool $\mathcal{T}$, evaluating c means invoking $\mathcal{T}$ and observing an outcome $y \in \{\text{success}(q), \text{fail}(\text{type})\}$, where q is a quality-of-result vector (area, latency) and type is an error category extracted from the tool log.

Two tools anchor our evaluation. **PandA-Bambu** [3] is a source-to-source HLS compiler with a statically scheduled datapath. Its pragma surface includes memory controller type (`MEM_ACC_11`, `MEM_ACC_N1`, `MEM_ACC_NN`), channel count, loop pipelining, and binding strategy. **Dynamatic** [4] generates dynamically scheduled (elastic dataflow) circuits and exposes a different surface: buffer placement, slack, fast-token paths, and target clock period. The two tools differ substantially in scheduling philosophy and failure mode distribution, making them a useful pair for stressing tool-generality claims.

Feasibility as a first-class outcome. A central observation motivating this work is that a large fraction of $\mathcal{C}$ produces outright synthesis failure, not merely poor-quality success. On Bambu with our chosen pragma ranges, two rule-provable incompatibility patterns alone account for 33.3% of the 420-point space: `MEM_ACC_11` requires exactly one channel, and `MEM_ACC_NN` requires at least two, so their union with the full channel-count range produces a deterministic infeasibility band. Beyond this, source-dependent patterns—pipelining a loop containing an accumulation dependence, or a data-dependent branch—drive further failures for a subset of benchmarks. A DSE method unaware of these structural failures will repeatedly consume budget on configurations the tool cannot synthesize.

B. Related Work

a) General-purpose optimizers for HLS DSE.: Simulated annealing, genetic algorithms, and Bayesian optimization are widely used as HLS DSE backends because they make minimal assumptions about the objective [2], [6], [7]. Their blind spot is that they treat synthesis failure as a dominated sample rather than as structural information. In our Bambu evaluation, simulated annealing reaches 18.4% SR and genetic algorithms reach 50.6% at B=60, with high variance that reflects their sensitivity to the fraction of infeasible configurations. Bayesian optimization with Gaussian processes [5] fares better (71.3%) because its surrogate implicitly learns the failure boundary, but the surrogate is slow to warm up on small budgets.

b) Learned surrogates and classifiers.: A more recent line of work trains machine-learning models to predict synthesis outcomes before calling the tool. Approaches range from per-benchmark regressors and reinforcement-learning agents [8] to graph neural networks operating on LLVM IR or custom HLS-specific graphs [9], [12], [13] to random forests in a scalable surrogate framework [1], [7]. Hybrid pipelines that combine learned surrogates with Bayesian optimization have also been proposed [10], [11]; HGBO-DSE [10] in particular uses a tree-structured modeler to prune invalid configurations before BO search, occupying a design point closest to ours. These methods can be effective, but they share two weaknesses. First, they require offline training, which in turn requires a labeled sample of the very design space they are meant to explore—often a chicken-and-egg problem for new tool versions or new benchmarks. Second, they produce point predictions whose failure modes are hard to interpret: when a surrogate confidently predicts success but the tool fails, debugging is difficult because the surrogate has no notion of *why* it failed. Our random-forest baseline (RF, 86.0% SR on Bambu) is a proxy for this family under our tool versions and budgets, not a full re-implementation of any specific prior system.

c) Meta-learning and cross-benchmark transfer.: Recent work has explored transfer and meta-learning to amortize DSE cost across benchmarks. These approaches are complementary to PA-DSE: they focus on *cross-benchmark* knowledge transfer, whereas PA-DSE operates *within a single run* and does not persist learned signatures across runs. A potential extension is to use PA-DSE-extracted signatures as meta-features for cross-run transfer.

d) Rule-based filtering.: Rule-based feasibility pruning has been explored in autotuning contexts outside of HLS. Closer to our setting, HGBO-DSE [10] prunes invalid configurations via a tree-structured modeler before Bayesian optimization. Our work differs in two respects: (i) PA-DSE's feasibility pruning operates at *two* evidence strengths (proven rules via SCF and online-learned signatures via RPE) rather than a single pre-search filter, and (ii) PA-DSE does not rely on an offline-trained GNN surrogate, trading some long-budget optimality for training-free, single-run operation.

e) Positioning.: PA-DSE occupies a different design point from all of the above: it is (i) training-free, (ii) single-run (no cross-run persistence of DFRL state), (iii) *decision-traceable* at every step (every skip is attributable to a rule or a signature), and (iv) near-zero-overhead (algorithmic cost $< 1\%$ of synthesis cost). Its goal is not to outperform all learned methods in the long-budget limit; it is to provide a robust, inspectable DSE policy that works well at the small budgets typical of interactive development.

III. PA-DSE Framework

A. Problem Setting and Policy Design

We frame feasibility-aware DSE as an *online budgeted exploration problem* and describe PA-DSE as a *deterministic hierarchical exploration policy* for this problem. We do not claim that PA-DSE solves an optimization problem in a formal sense; rather, we use the problem framing to clarify what the policy is trying to achieve and what trade-offs it makes.

TABLE I
PA-DSE Evidence Hierarchy

Evidence Source	Strength	Action	Scope
Tool-documented constraint	Proven	Permanent block	All runs
Source-code heuristic	Medium (empirical)	Queue ordering prior	All runs
Recurrent failure signature	High (empirical)	Hard skip	Current run
Per-dimension failure stats	Low–medium	Queue reordering	Current run

State. At step t, the observable state is $\mathcal{S}_t = (Q_t, \mathcal{H}_t, \mathcal{P}_t, \mathcal{W}_t)$, where Q_t is the remaining evaluation queue, $\mathcal{H}_t = \{(c_j, y_j)\}_{j<t}$ is the observation history with $y_j \in \{\text{success}, (\text{fail}, \text{type}_j)\}$, $\mathcal{P}_t$ is the current set of active RPE signatures, and $\mathcal{W}_t$ is the current OFRS statistics.

Actions. For each configuration $c \in Q_t$, the policy selects one of: Evaluate (consume one budget unit, observe outcome), Skip (permanently remove from Q_t), or Defer (reorder within Q_t).

Operational goals. The policy aims to:

1) Maximize the number of feasible designs discovered within a fixed budget B.
2) Minimize false skips—configurations incorrectly removed that were actually feasible.
3) Secondarily, minimize time-to-first-feasible and maximize best-QoR-under-budget.

These are *evaluation criteria* used to assess policy quality, not formal constraints enforced by the algorithm. In particular, the false skip rate is an *empirical safety target*, not an online guarantee: PA-DSE does not have a mechanism that provably bounds false skips below any threshold. Instead, it relies on the evidence hierarchy (Section III-B) and conservative activation rules to keep false skips low in practice.

Policy structure. PA-DSE implements a deterministic, hierarchical policy:

1) If c matches a sound blocking rule $\rightarrow$ Skip (pre-exploration, provably correct).
2) If c matches an active RPE signature with sufficient evidence $\rightarrow$ Skip (run-scoped, empirically validated).
3) Otherwise, rank c by OFRS risk score; evaluate lowest-risk first (Defer high-risk configs to end of queue).

B. Evidence Hierarchy

PA-DSE is organized around a single design principle: *action strength must not exceed evidence strength*. Table I summarizes the mapping. Only proven constraints (tool documentation) trigger permanent, cross-run blocking. RPE signatures trigger run-scoped hard skip based on empirical evidence accumulated during exploration. OFRS influences evaluation order but never permanently excludes any configuration.

C. Failure Taxonomy (Modeling Assumption)

Assumption. DFRL assumes that synthesis failures can be meaningfully partitioned into a small set of *normalized failure categories* (error types), and that configurations sharing the same error type share exploitable structural commonalities. If

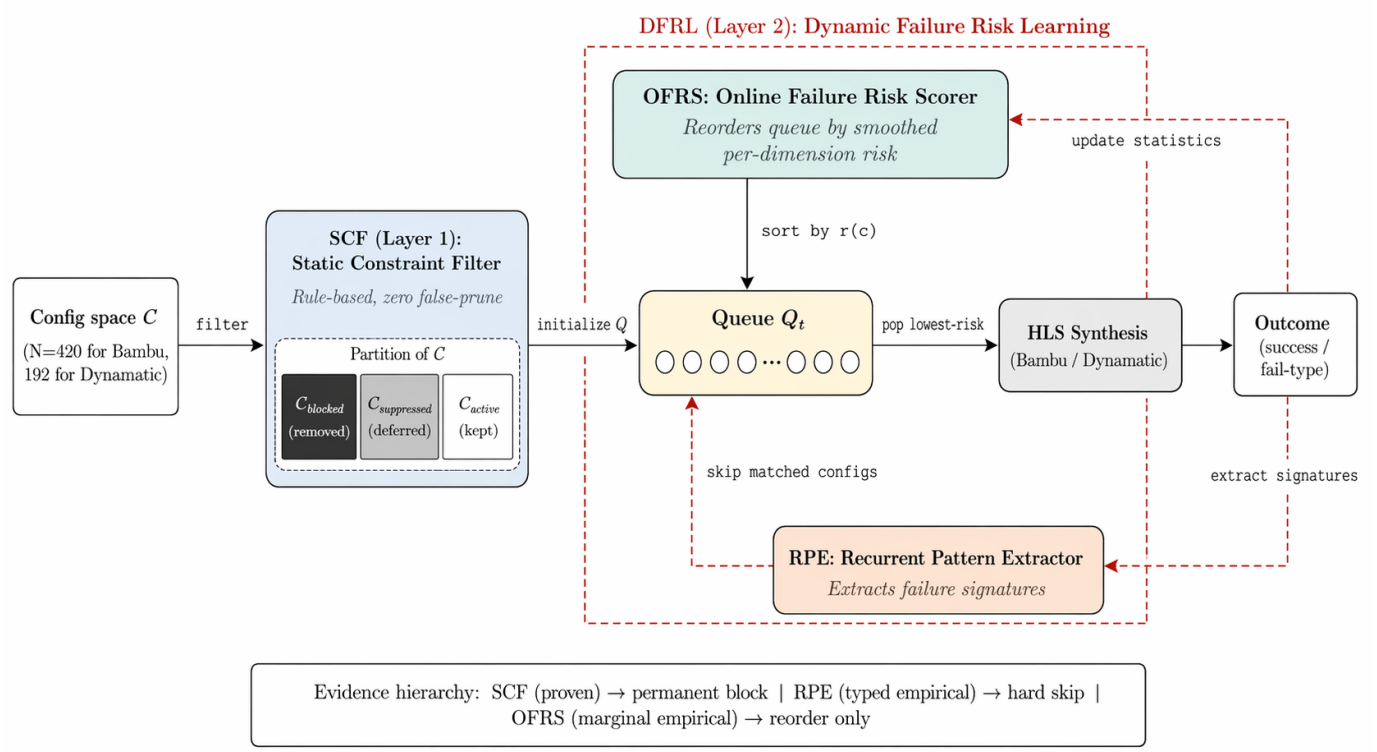

Fig. 1. PA-DSE architecture. Layer 1 (SCF) performs offline rule-based filtering and partitions the config space $\mathcal{C}$ into *blocked*, *suppressed*, and *active* subsets. Layer 2 (DFRL) operates online: OFRS reorders the remaining queue each iteration using smoothed per-dimension failure statistics; RPE extracts typed failure signatures from observed errors and removes matching configurations from the queue for the remainder of the run. Dashed red arrows denote the online feedback loop. Each component's action class (permanent block, hard skip, reorder) is strictly bounded by its evidence strength (bottom caption).

this assumption is violated—for example, if distinct failure mechanisms produce the same error type, or if a single mechanism produces inconsistent error types across runs—then RPE's per-type signature extraction will degrade. The validity of this assumption is empirical and tool-dependent.

Construction. We define a deterministic keyword-based mapping from raw tool log output to canonical error types. For Bambu: `pipeline_phi_conflict`, `generic_error`, `param_incompatibility`. For Dynamatic: `timeout`, `buffer_placement_failed`. The mapping is tool-specific but deterministic: the same log output always produces the same type.

Known limitations. Under-splitting (different root causes $\rightarrow$ same type) dilutes RPE's ability to find consistent dimensions. Over-splitting (one root cause $\rightarrow$ multiple types) fragments the failure evidence. The binary_search benchmark illustrates this: failures span `timeout` and `buffer_placement_failed`, limiting RPE generalization.

Stability verification. We verify cross-run consistency on a subset of 20 configurations per benchmark (same config $\rightarrow$ same error type across repeated runs) and manually spot-check 10 logs per type. These checks are necessary but not sufficient.

D. Layer 1: Static Constraint Filtering (SCF)

SCF partitions $\mathcal{C} = \mathcal{C}_{\text{blk}} \cup \mathcal{C}_{\text{sup}} \cup \mathcal{C}_{\text{act}}$ using a small set of hand-crafted rules, each labeled with a *severity* that determines the resulting action.

Sound blocking ($\mathcal{C}_{\text{blk}}$): Permanently removed based on tool documentation (Bambu R1–R2). Provably correct; zero false-positive rate. Within the evidence hierarchy, this is the strongest action, reserved for proven constraints.

Heuristic suppression ($\mathcal{C}_{\text{sup}}$): Provides an *initial queue ordering prior*—suppressed configurations are placed after active ones in the initial queue. This is not a hard exclusion: if OFRS later assigns a suppressed configuration a lower risk score than some active configurations, the suppressed configuration may be promoted ahead of them. Suppression establishes a starting bias that can be overridden by online evidence. If budget permits, all suppressed configurations are eventually evaluated.

E. Layer 2: Dynamic Failure Risk Learning (DFRL)

DFRL operates online during exploration, updating its internal state after each synthesis evaluation. It is not a classifier, surrogate model, or machine-learning method: no offline training, no gradient updates, no model fitting. Its state is fully deterministic given $\mathcal{H}_t$ and inspectable at any point.

DFRL requires the failure taxonomy assumption (Section III-C) to hold for RPE, and sufficient observation volume for OFRS.

1) Recurrent Pattern Extractor (RPE): **Semantic scope.** RPE outputs are *typed recurrent failure signatures*: parameter conditions that consistently co-occur with a specific error type and discriminate failures from successes in the current obser-

vation history. They are discriminative summaries, *not* root-cause identifications. Their validity depends on observation coverage and taxonomy stability.

Within the evidence hierarchy (Section III-B), RPE operates at the *typed recurrent evidence* level: it requires concentrated, type-specific failure observations to trigger hard skip—the second-strongest action after sound blocking. This positioning reflects RPE's reliance on structured, repeated evidence rather than marginal or mixed-type statistics.

Signature definition. A signature is a tuple $\pi = (t, \mathbf{cond})$ where t is a normalized error type and $\mathbf{cond} = \{(d_1, v_1), \ldots, (d_k, v_k)\}$ is a set of dimension-value pairs. A configuration c *matches* π, written $c \models \pi$, if and only if $c[d_i] = v_i$ for all $(d_i, v_i) \in \mathbf{cond}$.

The signature's evidence counts are defined over the *entire* current observation history $\mathcal{H}_t$:

$$n_{\text{fail}}(\pi) = |\{(c, y) \in \mathcal{H}_t : y = (\text{fail}, t) \text{ and } c \models \pi\}| \quad (1)$$

$$n_{\text{counter}}(\pi) = |\{(c, y) \in \mathcal{H}_t : y = \text{success and } c \models \pi\}| \quad (2)$$

That is, n_{fail} counts all observed type-t failures matching π, and n_{counter} counts all observed successes matching π, regardless of when they were observed.

Activation boundary. RPE attempts signature extraction only when: (1) $\geq \tau$ failures of the same error type t exist in $\mathcal{H}_t$, and (2) ≥ 1 successful evaluation exists in $\mathcal{H}_t$ (providing contrast for discriminative filtering).

Signature formation. RPE attempts to identify, for each error type, a compact, non-redundant set of parameter conditions that captures the recurrent structure of same-type failures while excluding coincidentally shared but non-informative dimensions. The formation procedure operates as follows:

Discriminative dimension filtering. Given the set of all type-t failures in $\mathcal{H}_t$:

1) **Consistency:** For each dimension d, check whether all type-t failures share the same value v. Retain only dimensions where this holds.
2) **Discrimination:** Among retained dimensions, exclude any where $d = v$ also holds for *every* observation in $\mathcal{H}_t$ (both failures and successes). Such dimensions carry no discriminative information.
3) **Scoring:** The surviving dimensions form a candidate signature $\pi = (t, \mathbf{cond})$ with an evidence score:

$$\text{conf}(\pi) = \frac{n_{\text{fail}}(\pi)}{n_{\text{fail}}(\pi) + n_{\text{counter}}(\pi) + 2} \quad (3)$$

The additive constant $+2$ provides Laplace smoothing, ensuring the score is well-defined even with zero counterexamples. This score is a *smoothed empirical evidence score*—not a calibrated probability, not a formal posterior estimate, and not a guaranteed risk bound. It summarizes the concentration of same-type failure evidence relative to counterevidence, and serves as a conservative activation gate: monotonically increasing in n_{fail}, monotonically decreasing in n_{counter}.

Activation threshold. A signature is activated (triggers hard skip for matching configs) only when $\text{conf}(\pi) \geq \theta$, with $\theta = 0.8$ in our experiments. To give concrete intuition: with $\theta = 0.8$, a signature with zero counterexamples requires $n_{\text{fail}} \geq 8$ to activate $(8/(8+0+2) = 0.8)$. With one counterexample, it requires $n_{\text{fail}} \geq 12$ $(12/(12+1+2) = 0.8)$. This means hard skip is triggered only after substantial and consistent failure evidence. The threshold θ is a user-configurable parameter; sensitivity analysis across $\theta \in \{0.7, 0.8, 0.9\}$ is reported in the experiments.

Scope and lifecycle. Signatures are *run-scoped*: they apply only within the current exploration run, not across runs or benchmarks. Within a run, signatures follow *threshold-triggered activation with persistence*: once $\text{conf}(\pi) \geq \theta$, the signature becomes active and is applied to all remaining queue members. The signature remains active as long as $\text{conf}(\pi) \geq \theta$. Since evidence counts are computed over the entire history $\mathcal{H}_t$ (Eqs. 1–2), a signature's confidence can decrease if subsequent observations add to n_{counter}—either through configurations evaluated before the signature was formed, or through probe auditing (below). If $\text{conf}(\pi)$ drops below θ, the signature is deactivated and no longer triggers hard skip.

The lifecycle follows three rules: (1) *activation* is threshold-triggered: a signature becomes active when $\text{conf}(\pi) \geq \theta$; (2) *persistence*: an active signature remains in effect as long as $\text{conf}(\pi) \geq \theta$; (3) *deactivation* occurs only through observed counterevidence that increases n_{counter} and drives $\text{conf}(\pi)$ below θ. Hard-skipped configurations do not directly generate counterevidence because they are not evaluated. The only two sources of counterevidence are: (a) configurations evaluated before the signature was formed that happen to match it and succeeded, and (b) probe evaluations of would-be-skipped configurations.

Probe auditing. To provide a self-correction path, a configuration that would be hard-skipped is instead evaluated with probability p_{probe} (default 0.05). If the probe succeeds, n_{counter} increases and $\text{conf}(\pi)$ decreases, potentially deactivating the signature. This costs at most $\lceil p_{\text{probe}} \cdot |\text{skipped}| \rceil$ additional budget units.

Subsumption. Signature $\pi_1 = (t, \mathbf{cond}_1)$ *subsumes* $\pi_2 = (t, \mathbf{cond}_2)$ if: (a) they share the same error type t; (b) $\mathbf{cond}_1 \subsetneq \mathbf{cond}_2$ (π_1 has strictly fewer conditions, all of which appear in $\mathbf{cond}_2$); and (c) π_1 is itself *activated* ($\text{conf}(\pi_1) \geq \theta$). When all three conditions hold, π_2 is deactivated and removed from $\mathcal{P}_t$, because π_1 matches a strict superset of the configurations matched by π_2 and has sufficient evidence to justify hard skip on its own. Condition (c) is critical for conservativeness: a newly formed but insufficiently supported general candidate must not displace an already-activated narrower signature.

2) Online Failure Risk Scorer (OFRS): OFRS is a *deliberately simple*, first-order, small-sample risk ranking model. Its sole function is queue reordering: delaying configurations with higher estimated failure association, never permanently excluding any configuration. It is not a calibrated failure predictor, probability estimator, or classifier.

Within the evidence hierarchy (Section III-B), OFRS operates at the weakest evidence level: per-dimension marginal statistics that mix across failure types. This positioning dictates that OFRS may only reorder, never permanently exclude—the weakest action available.

Why OFRS only reorders, never skips. Unlike RPE, which operates on concentrated, high-evidence per-type sig-

natures, OFRS aggregates marginal single-dimension statistics across all failure types. These marginal statistics are: (1) inherently noisy at small sample sizes; (2) unable to capture parameter interactions that may drive failure; (3) not type-specific, so they mix evidence from structurally different failure modes. Granting OFRS skip authority would allow noisy, marginal evidence to permanently exclude configurations—including potentially Pareto-optimal feasible ones. Restricting OFRS to reordering ensures that all configurations remain reachable if budget permits, and that the only permanent exclusion mechanism (RPE) requires type-specific, concentrated, high-evidence signatures.

Why first-order is sufficient here. HLS infeasibility can involve parameter interactions. A higher-order model could in principle capture these, but at budgets $B \leq 80$, most pairwise $(d_1{=}v_1, d_2{=}v_2)$ combinations have 0–2 observations, making pairwise statistics unreliable. RPE handles interaction effects implicitly through its consistency-based extraction (a signature like {pipeline=True} effectively captures all interactions involving pipeline). OFRS is designed to provide marginal ranking improvement as a complement to RPE, not to model failure structure independently. In interaction-dominated scenarios where no single dimension separates failure from success, all $w_d \approx 0$ and OFRS produces near-uniform rankings, gracefully degrading to no-op.

Formulation. For each dimension d and value v, the smoothed failure association score is:

$$\hat{f}(d, v) = \frac{n_f(d{=}v) + 1}{n_f(d{=}v) + n_s(d{=}v) + 2} \tag{4}$$

where $n_f(d{=}v)$ and $n_s(d{=}v)$ are the number of observed failures and successes, respectively, with $c[d] = v$. We use $\hat{f}$ rather than $\hat{P}$ to emphasize that this is a smoothed empirical score, not a calibrated probability estimate.

The *relevance* of dimension d is the range of $\hat{f}$ across its observed values:

$$w_d = \max_v \hat{f}(d, v) - \min_v \hat{f}(d, v) \tag{5}$$

with cold-start protection: $w_d = 0$ if total observations for dimension d are below $n_{\min} = 5$.

The risk score for configuration c is the relevance-weighted average:

$$r(c) = \begin{cases} \dfrac{\sum_d w_d \cdot \hat{f}(d, c[d])}{\sum_d w_d} & \text{if } \sum_d w_d > 0 \\ 0.5 & \text{otherwise (cold-start fallback)} \end{cases} \tag{6}$$

When $\sum_d w_d = 0$, OFRS does not alter the queue order; configurations retain the ordering established by SCF. As observations accumulate and individual dimensions cross the $n_{\min}$ threshold, OFRS gradually transitions from this passive mode (preserving the static prior) to active global reordering. This transition is smooth: each dimension independently contributes to ranking only after it has accumulated sufficient evidence, so OFRS's influence grows incrementally rather than switching on abruptly.

F. Module Interface Specification

The two layers yield four distinct operations that interact through a well-defined priority ordering, each reflecting the evidence hierarchy principle that action strength must not exceed evidence strength:

1. Sound blocking (SCF) permanently removes configurations from the design space before exploration begins. This action is irreversible and provably correct (zero false positives). *Evidence: proven constraints. Action: strongest (permanent block).*

2. Heuristic suppression (SCF) establishes an *initial queue ordering prior*: suppressed configurations are placed after active ones. This is *not* a hard exclusion—it is an ordering bias that OFRS may override as evidence accumulates. In practice, suppressed configurations typically receive high risk scores from OFRS (because they match failure-prone parameter combinations), so OFRS tends to reinforce rather than contradict the static prior. *Evidence: medium (empirical). Action: ordering bias, overridable.*

3. RPE hard skip removes matching configurations from the queue during exploration. RPE operates independently of OFRS: it removes configs before OFRS ranks the remainder. A configuration may be simultaneously suppressed (by SCF) and RPE-skipped; the skip takes precedence. *Evidence: high (empirical, typed, concentrated). Action: run-scoped permanent removal.*

4. OFRS reordering sorts the remaining queue by risk score. This is a global sort over all remaining configurations—both active and suppressed. OFRS reordering is the weakest action: it never removes configurations, only changes their evaluation order. *Evidence: low–medium (marginal, mixed-type). Action: weakest (reorder only).*

Conflict handling. OFRS may promote a heuristically-suppressed configuration ahead of active ones if its risk score is lower. This is by design: OFRS reacts to actual synthesis evidence, which may reveal that a suppressed configuration's non-suppression parameters are associated with success. Since OFRS only promotes (evaluates earlier, not skips), the worst case is one wasted budget unit, which is always preferable to permanently excluding a potentially feasible design.

Cold-start window. During the first $\sim n_{\min}$ evaluations, OFRS has insufficient data for meaningful relevance scores. During this period, queue order follows the SCF prior (active before suppressed). RPE also requires $\geq \tau$ same-type failures before activating, so early exploration proceeds without dynamic intervention.

G. Exploration Algorithm

Algorithm 1 summarizes the PA-DSE policy. Newly learned RPE signatures take effect in the next iteration of the while loop, not retroactively.

IV. Experimental Setup

A. Tools and Benchmarks

We evaluate PA-DSE on two HLS tools that differ fundamentally in scheduling philosophy:

Algorithm 1 PA-DSE Exploration Policy

Require: $\mathcal{C}$, source S, budget B, threshold τ, probe rate p_{probe}
Ensure: Results $\mathcal{R}$

```
C_act, C_blk, C_sup ← SCF(C, S)
Q ← C_act ∥ C_sup; n ← 0
while Q ≠ ∅ and n < B do
    for c ∈ Q do
        if RPE.matches(c) then
            if rand() < p_probe then
                mark c as probe
            else
                remove c from Q {Hard skip}
            end if
        end if
    end for
    Sort Q by r(c) ascending {OFRS: global reorder}
    c ← Q.popFirst(); n ← n + 1
    (ok, met, out) ← Synthesize(c, S)
    if ok then
        OFRS.update(c, success)
        if c is probe then
            RPE.addCounter(c)
        end if
    else
        OFRS.update(c, failure)
        RPE.tryExtract(out, c, τ)
    end if
    R ← R ∪ {(c, met, ok)}
end while
return R
```

- **PandA-Bambu v0.9.8** [3]: statically scheduled, C-centric, with a well-established failure-taxonomy surface.
- **Dynamatic v2.0** [4]: dynamically scheduled (elastic dataflow), LLVM-based, with MILP-driven buffer placement (solved using Gurobi 12.0 in our setup).

This pairing stresses tool-generality: the two tools share no pragma vocabulary, expose disjoint failure modes, and benefit asymmetrically from static versus dynamic pruning, as we analyze below.

Benchmarks. Eight benchmarks for Bambu (`matmul`, `vadd`, `fir`, `histogram`, `atax`, `bicg`, `gemm`, `gesummv`) and four for Dynamatic (`gcd`, `matching`, `binary_search`, `kernel_2mm`). The selection combines Polybench linear-algebra kernels, simple DSP and control-flow kernels, and ML-style reductions, covering a range of loop-nest structures and feasibility landscapes.

B. Configuration Spaces

Bambu (420 configurations). The enumerated space is the Cartesian product of memory controller choices (`channels_type` $\in$ {`MEM_ACC_11`, `MEM_ACC_N1`, `MEM_ACC_NN`}), channel count (`channels_number` $\in \{1, 2, 4\}$), target clock period, loop pipelining on/off, and further binding/scheduling options. Two tool-documented incompatibilities contribute a provably infeasible *blocked* subset $\mathcal{C}_{\text{blk}}$ of size 140 (33.3%): MEM_ACC_11 requires exactly one channel, MEM_ACC_NN requires at least two. In addition, source-level heuristics (pipelining interacting with accumulation or control-flow patterns) mark up to 224 configurations as *suppressed* for a subset of benchmarks (`matmul`, `fir`, `histogram`); these are deferred but not removed from the queue.

Dynamatic (192 configurations). The enumerated space covers target clock period, buffer algorithm (`on_merges`, `fpga20`, `fpl22`), hardware sharing, LSQ sizing, and fast-token-delivery tuning. *No tool-documented incompatibility rules apply to this space,* so under SCF all 192 configurations remain in the active set: $\mathcal{C}_{\text{blk}} = \emptyset$ and $\mathcal{C}_{\text{sup}} = \emptyset$. SCF-on and SCF-off behave identically on Dynamatic, making this tool a deliberate stress test of whether DFRL can carry the feasibility-aware load on its own (Section V-G).

C. Baselines

We compare against six baselines chosen to span the classical and learned HLS DSE landscape:

- **Random**: uniform sampling without replacement from $\mathcal{C}$.
- **Filtered Random**: uniform sampling from $\mathcal{C} \setminus \mathcal{C}_{\text{blk}}$—a strict SCF-only baseline. On Dynamatic this reduces to Random since $\mathcal{C}_{\text{blk}} = \emptyset$.
- **Simulated Annealing (SA)** and **Genetic Algorithm (GA)**: classical black-box optimizers using feasibility as fitness [6], [7].
- **Bayesian Optimization (GP-BO)**: Gaussian-process surrogate with expected-improvement acquisition, following the BO-for-HLS-DSE line of work [2], [5].
- **Random Forest Classifier (RF)**: an online-trained feasibility classifier modelled after the scalable surrogate of AutoScaleDSE [1], using confidence-thresholded sampling. We use it as a *proxy baseline* for the learned-classifier family under our configuration space and budgets, not as a full re-implementation.

All baselines share the same $\mathcal{C}$, the same synthesis interface, and the same per-run budget. We did not compare against offline-trained learned surrogates from prior work [8], as doing so would require retraining on our specific tool versions and configuration space; the RF baseline serves as a close proxy.

D. Evaluation Protocol

Budgets. We use B=60 evaluations per run on Bambu and B=30 on Dynamatic. These are chosen to represent realistic "coffee-break" interactive budgets: with per-call synthesis cost of $\approx$ 2.5 s (Bambu) and $\approx$ 6 s (Dynamatic), a full run completes in 2–3 minutes of wall clock in both cases. We also sweep $B \in \{30, 60\}$ in the Bambu ablation.

Repetitions. Each (method, benchmark, budget) cell is repeated 10 times. For stochastic baselines (Random, SA, GA, GP-BO, RF) the repetition index is a random seed. For deterministic methods (Filtered Random, PA-DSE) we vary the initial queue ordering via a `queue_permutation_id`, which is mechanically equivalent to a seed for sensitivity analysis. This isolates ordering sensitivity from optimizer-internal randomness.

PA-DSE hyperparameters. We freeze τ=2 (minimum failure support for RPE signature activation), θ=0.8 (RPE confidence threshold), $n_{\min}$=5 (OFRS cold-start observation threshold), and p_{probe}=0.05 (probe rate for keeping active signatures honest). These defaults are not tuned per-benchmark or per-tool.

Metrics. We report:

- **SR (%)**: fraction of evaluated configurations that synthesized successfully—the primary feasibility metric.
- **Wasted calls**: count of synthesis invocations that returned a failure—a proxy for user-visible time cost.
- **TTFF (s)**: wall-clock time to first feasible design, relevant for interactive use.
- **UQoR**: number of unique Pareto-eligible QoR points (distinct (area, latency) pairs) discovered.
- **Best area, best latency**: minima observed across successful configurations in the run.
- **Algorithmic overhead**: per-iteration time spent in each PA-DSE layer (`overhead_phago_ms`, `overhead_rpe_ms`, `overhead_ofrs_ms`).

Hardware. All runs execute single-threaded on an Azure `Standard_D4s_v5` VM (4 vCPUs, 15 GB RAM, Ubuntu 24.04, Python 3.12). No GPUs are used. Gurobi 12.0 (academic license) is used for Dynamatic's MILP buffer placement.

Reproducibility. Code, configuration generators, benchmark sources, raw run logs, and all aggregation scripts are released at https://github.com/jane09250410/hls-dse. A single `run_all.sh` regenerates every table and figure in this paper.

V. Results

We structure the evaluation around six questions, each addressed in its own subsection: (1) Does PA-DSE outperform classical and learned baselines on success rate? (2) How large is the user-visible cost savings in terms of wasted calls and TTFF? (3) Are per-benchmark results consistent, or driven by a few easy benchmarks? (4) What does the convergence trajectory look like—does PA-DSE reach its SR early or late in the budget? (5) Does PA-DSE's queue reordering sacrifice QoR quality? (6) Does the ablation confirm the evidence-hierarchy design principle? (7) Is the algorithmic overhead of PA-DSE negligible relative to synthesis cost?

A. Main Comparison on Bambu

Table II reports per-method means and standard deviations across all $(benchmark, \text{run})$ cells on Bambu at B=60. PA-DSE achieves 92.5%±1.1 success rate, exceeding the strongest baseline (RF, 86.0% ± 1.1) by 6.5 pp. More importantly, PA-DSE wastes only 4.5 synthesis calls on average per run, compared to 8.4 for RF, 17.2 for GP-BO, and 51 for Random—a **2**× reduction over the best baseline and a **11**× reduction over Random. Because each wasted call costs ≈ 2.5 s of user time on Bambu, PA-DSE saves roughly 10 s of wall clock per run versus RF.

Two additional observations are worth noting. First, PA-DSE's variance is the lowest of any method with non-trivial SR (σ=1.1), tied with RF. Simulated annealing, by contrast, has σ=29.4—some runs find feasible configurations early and succeed, while others get trapped in infeasible regions; this reflects a broader pattern that classical black-box methods are ill-suited to design spaces dominated by structural infeasibility. Second, PA-DSE matches or beats all baselines on QoR quality (best area, best latency, UQoR): reliability does not trade off against exploration quality. Figure 2 visualizes the full per-method distribution.

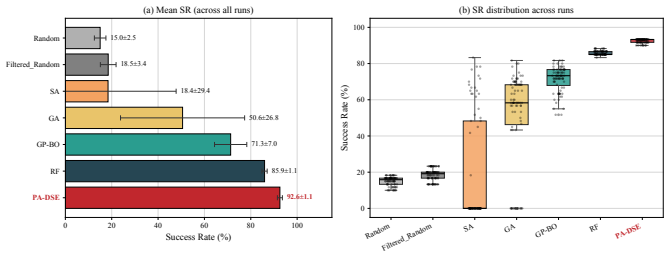

Fig. 2. Bambu main results (B=60). (a) Mean SR with standard deviation error bars. (b) Per-run SR distribution: PA-DSE has the tightest distribution of any method with non-trivial SR.

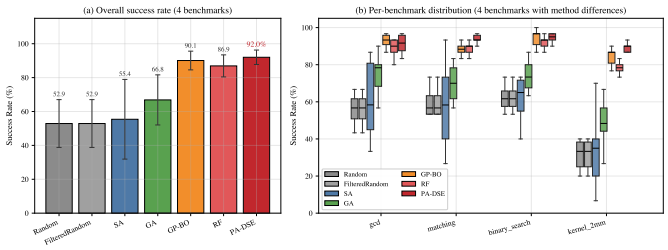

Fig. 3. Dynamatic main results (B=30). (a) Aggregate SR with standard deviation across four benchmarks. (b) Per-benchmark SR distribution. PA-DSE matches or exceeds all baselines across benchmarks with substantially lower variance.

B. Cross-Tool Evaluation on Dynamatic

Table III reports results on Dynamatic at B=30, aggregated over the four evaluation benchmarks. Because Dynamatic exposes no tool-documented incompatibility rules ($\mathcal{C}_{\text{blk}} = \emptyset$), SCF reduces to a no-op—Filtered Random is identical to Random, and the SCF-only ablation matches no-filter (Section V-G). Any improvement over Random must therefore come from DFRL.

PA-DSE achieves 92.0% SR, narrowly ahead of GP-BO (90.1%) but with a distinctive TTFF advantage: 26.5 s compared to ≈ 50 s for GP-BO and RF. For interactive use—where a developer waits for the first feasible design before beginning inspection—PA-DSE reaches that first point in roughly half the wall-clock time. The UQoR difference (PA-DSE 7.1 vs. Random 8.8) reflects a tradeoff in QoR diversity discussed in Section V-F: PA-DSE reaches all Pareto-optimal points but covers fewer Pareto-dominated interior points because its reordering concentrates the budget on low-risk regions. Figure 3 shows the per-benchmark distribution.

C. User-Visible Cost: Wasted Calls and Time-to-First-Feasible

Success rate is only one dimension of user experience; two others dominate perceived quality in interactive DSE: how

TABLE II
MAIN RESULTS ON BAMBU (B=60, n=80 RUNS PER METHOD, n=88 FOR PA-DSE). LOWER IS BETTER FOR WASTED CALLS, TTFF, AND BEST AREA/LATENCY; HIGHER IS BETTER FOR SR AND UQOR. PA-DSE LEADS ON EVERY RELIABILITY METRIC WHILE MATCHING THE BASELINES ON QUALITY.

Method	SR (%)	Wasted	TTFF (s)	UQoR	Best area	Best latency
Random	15.0 ±2.5	51.0	22.9	6.4	473.1	10.94
Filtered Random	18.5 ±3.4	48.9	11.9	7.9	473.9	10.94
SA	18.4 ±29.4	48.9	2.5	4.3	464.1	10.58
GA	50.6 ±26.8	29.7	7.7	11.8	464.1	10.61
GP-BO	71.3 ±7.0	17.2	10.9	14.9	464.1	10.28
RF	86.0 ±1.1	8.4	14.3	17.1	464.1	10.31
PA-DSE	**92.5** ±1.1	**4.5**	8.9	**19.1**	**464.1**	**10.25**

TABLE III
MAIN RESULTS ON DYNAMATIC (B=30, n=40 RUNS PER METHOD). FILTERED RANDOM = RANDOM BECAUSE SCF HAS NO APPLICABLE RULES ON DYNAMATIC. PA-DSE STILL LEADS ON SR AND TTFF, SHOWING THAT DFRL ALONE IS SUFFICIENT FOR TOOLS WITHOUT STATIC CONSTRAINTS.

Method	SR (%)	Wasted	TTFF (s)	UQoR	Best area	Best latency
Random	52.9 ±14.1	14.1	29.5	8.8	49.5	502.7
Filtered Random	52.9 ±14.1	14.1	29.5	8.8	49.5	502.7
SA	55.4 ±23.5	13.4	51.3	7.2	50.3	508.3
GA	66.8 ±14.8	10.0	29.5	8.3	49.5	503.3
GP-BO	90.1 ±5.5	3.0	50.1	7.7	49.4	511.3
RF	86.9 ±6.5	3.9	50.2	8.6	49.3	511.7
PA-DSE	**92.0** ±4.3	**2.4**	**26.5**	7.1	49.7	513.3

many synthesis calls are wasted (each wasted call is a direct time cost) and how quickly the first feasible design is found (influences iteration speed). Figure 4 summarizes both.

PA-DSE wastes 4.5 synthesis calls per run on Bambu ($11.3\times$ fewer than Random, $1.9\times$ fewer than RF) and 2.4 on Dynamatic ($5.9\times$ fewer than Random). Translated to wall-clock: on Bambu, PA-DSE saves about $10\,$s per run versus RF (≈ 4 calls $\times$ $2.5\,$s); on Dynamatic, about $9\,$s per run versus GP-BO. For interactive developers iterating tens of times a day, this compounds to meaningful time savings. TTFF shows a similar picture on Dynamatic ($26.5\,$s for PA-DSE vs. $\approx 50\,$s for GP-BO/RF); on Bambu, classical methods have unusually low TTFF because they find occasional trivial successes early, but their mean SR is much lower (Table II).

D. Per-Benchmark Consistency

Aggregate SR can mask method-benchmark interactions: a method might win on average while failing badly on a specific benchmark. Figure 5 plots mean SR for every (method, benchmark) cell.

PA-DSE achieves $\geq 90\%$ SR on every Bambu benchmark, while every baseline (including RF) has at least one benchmark where it drops noticeably. SA and GA in particular show high per-benchmark variance, consistent with their sensitivity to the benchmark-specific feasibility landscape. On Dynamatic the per-benchmark advantage is narrower—most methods stay within a 5–10 pp band of PA-DSE—but PA-DSE is the only method without a single benchmark where SR drops below 88%.

E. Convergence Behavior

Figure 6 shows the cumulative number of feasible designs found as a function of the evaluation step, averaged across runs. This reveals not just the *amount* of budget used but the *trajectory*: does PA-DSE front-load its successes or find them throughout?

On Bambu (panel a), PA-DSE's curve rises nearly linearly through the full 60-step budget, reflecting that DFRL continually surfaces feasible configurations rather than finding them all early and then stalling. RF catches up toward the tail; SA and GA exhibit characteristic stagnation. On Dynamatic (panel b), PA-DSE converges within the first ~ 20 steps to ≈ 28 feasibles, while classical methods (< 20 feasibles at step 30) never catch up.

F. Quality of Results (QoR)

Finding feasible configurations is necessary but not sufficient; users ultimately want coverage of the (area, latency) quality space, especially near the Pareto front. We evaluate QoR coverage by pairwise comparison: for each baseline

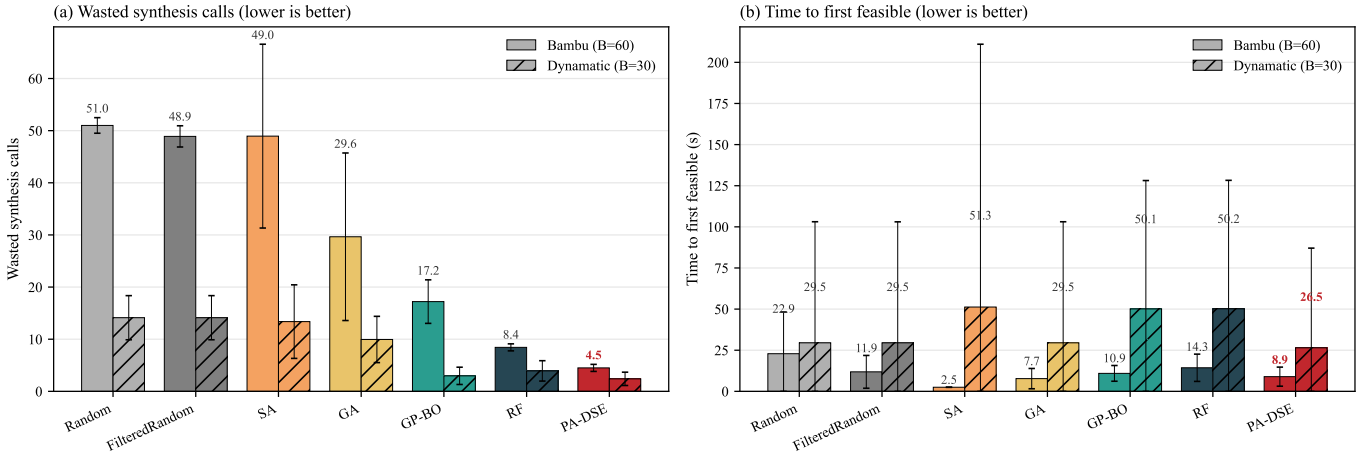

Fig. 4. User-visible cost. (a) Wasted synthesis calls per run (lower is better): PA-DSE wastes 4.5 on Bambu and 2.4 on Dynamatic, a 1.9×–11× reduction over other methods. (b) Time to first feasible (lower is better): on Dynamatic, PA-DSE reaches the first feasible design in half the wall-clock time of GP-BO and RF.

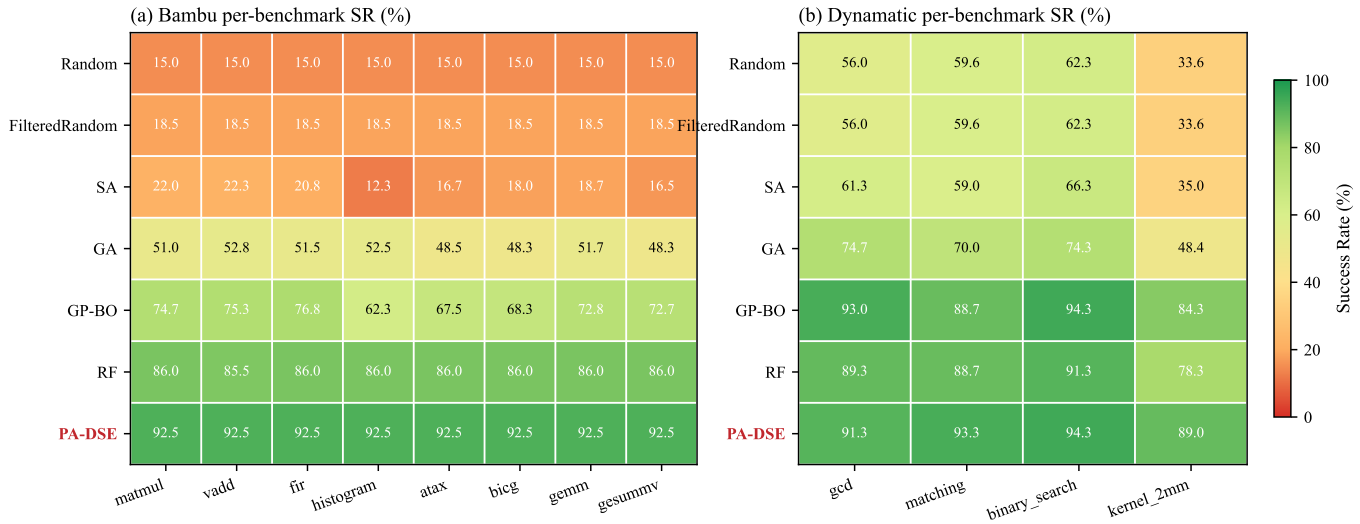

Fig. 5. Per-benchmark success rate (%). PA-DSE (bottom row, red label) wins or ties on every Bambu benchmark and every Dynamatic benchmark, with the tightest per-run variance of any method (see Section V-A).

separately, we count how many unique (area, latency) points are found by PA-DSE only, by that baseline only, or by both. This comparison is fair because each baseline and PA-DSE use a similar number of independent runs (80 and 88 respectively on Bambu; 40 each on Dynamatic). Figure 7 shows the result as a stacked bar for each pair; the number above each bar reports PA-DSE's *net advantage* (PA-DSE-only minus baseline-only).

Three observations follow. First, PA-DSE's net advantage is non-negative against every baseline on both tools: against weaker methods (Random, FilteredRandom, SA, GA) the advantage is substantial, and against the strongest learned baselines (GP-BO, RF) it remains positive or ties. The *shared* bands are large, confirming that PA-DSE rediscovers most of what any single baseline finds. Second, the *baseline-only* bands are consistently small, indicating that PA-DSE's feasibility focus does not create systematic blind spots in the QoR space. Third, PA-DSE reaches every Pareto-optimal point on both tools (not separately plotted; see Section VI). Reliability and QoR coverage are not trading against each other in our setting.

G. Ablation: Isolating the Contribution of Each Component

Table IV reports an 8-way ablation at $B{=}30$ on both tools, separating SCF, RPE, and OFRS and probing the effect of changing OFRS's action class from *reorder* to *skip*. Figure 8 visualizes the same data as a grouped bar chart. Several findings emerge:

a) OFRS is the dominant contributor to SR.: On Bambu, adding OFRS to SCF drives SR from 17.8% (SCF-only) to 84.5% (SCF+OFRS), a +66.7 pp gain. On Dynamatic the effect is similar: +38.7 pp from SCF-only (50.8%) to SCF+OFRS (89.5%). Across both tools, OFRS's online queue reordering is doing the heavy lifting at these budgets.

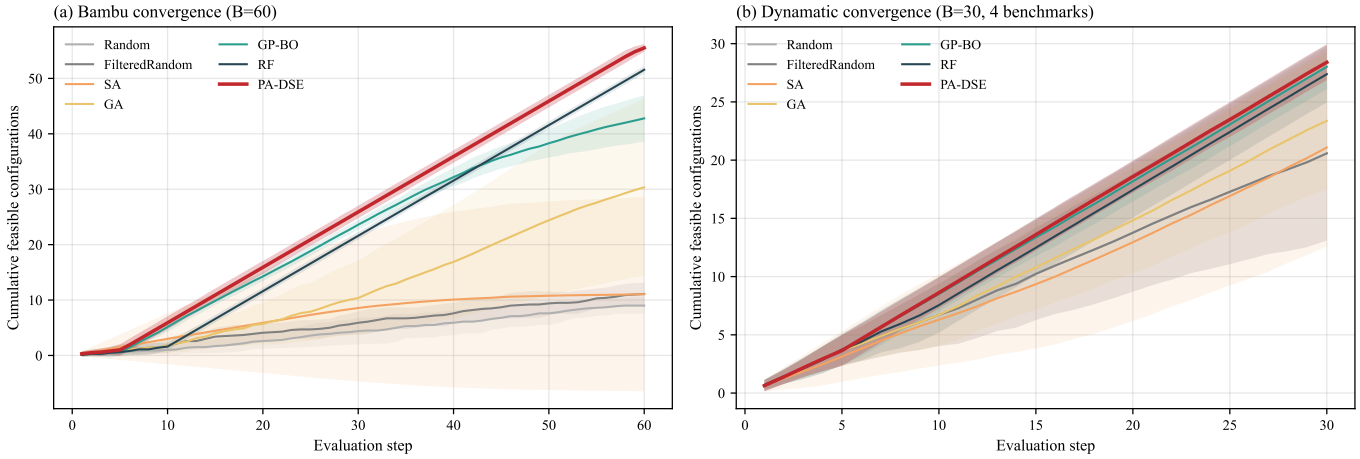

Fig. 6. Convergence curves (mean $\pm 1\sigma$ across runs). (a) Bambu at B=60: PA-DSE maintains near-linear progression throughout the budget, indicating that OFRS's online reordering keeps promising configurations at the queue front. (b) Dynamatic at B=30: PA-DSE's curve is visibly steeper than the classical baselines but converges with GP-BO/RF by mid-budget.

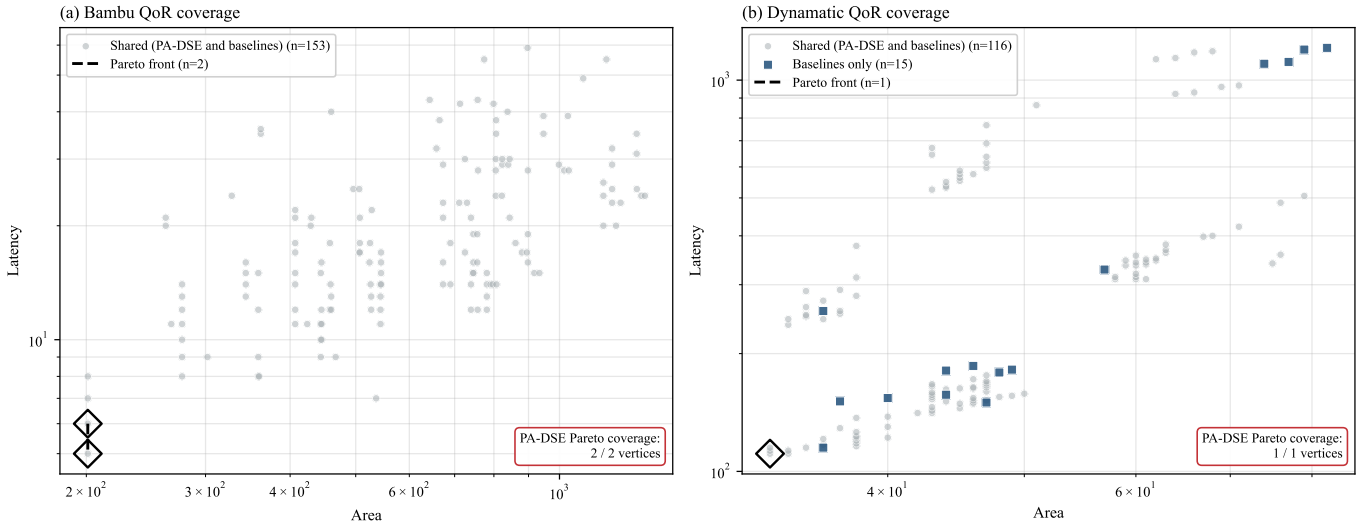

Fig. 7. Pairwise QoR coverage: PA-DSE vs. each baseline, independently. Each stacked bar shows, for the union of unique (area, latency) points discovered by PA-DSE and that single baseline: the shared subset (gray), points found only by the baseline (blue), and points found only by PA-DSE (red). The number above each bar is PA-DSE's net advantage (red − blue). PA-DSE achieves a non-negative net advantage against every baseline on both tools.

b) SCF provides a safety net on tools with static rules.: Comparing SCF+DFRL to DFRL-only isolates SCF's marginal value. On Bambu, SCF adds $+4.5$ pp ($80.0\% \rightarrow 84.5\%$) by removing provably infeasible configurations before DFRL can be contaminated by their failures. On Dynamatic, SCF adds 0 pp because $\mathcal{C}_{\text{blk}} = \emptyset$. This asymmetry confirms the design stance: SCF's benefit is contingent on the tool exposing documented rules, but it is non-negative—it never hurts.

c) RPE is absorbed by OFRS under our budgets.: SCF+DFRL and SCF+OFRS are numerically identical (84.5% on Bambu, 89.5% on Dynamatic). Yet SCF+RPE alone achieves only 57.8% on Bambu, well below SCF+OFRS. Inspection of per-run logs reveals the mechanism: when OFRS is active, it reorders configurations matching heuristically-suppressed patterns to the tail of the queue. These are precisely the configurations RPE would have extracted signatures from. At B=30, OFRS's reordering means the budget is exhausted before the queue advances to where RPE would activate. The components are functional; they are not in use at this budget. We expect RPE to contribute more at larger budgets or in failure-mode-diverse workloads; we leave this to future work.

d) RPE-as-reorder degrades to SCF+OFRS.: Swapping RPE's action from *skip* to *reorder* yields identical results to SCF+OFRS (84.5% on Bambu). This is not a contradiction: when OFRS is already reordering by failure risk, adding RPE's risk-based reorder is redundant. The ablation confirms that RPE's distinct contribution is its *skip* action, not any reordering signal.

e) Allowing OFRS to skip catastrophically increases variance.: The SCF+OFRS-as-skip configuration, which permits OFRS to hard-skip configurations at a risk threshold of 0.85, produces mean SR 28.9% with σ=41.8 on Bambu and 24.2% with σ=43.7 on Dynamatic. These variances are an order of magnitude larger than any other configuration.

TABLE IV
ABLATION ON BOTH TOOLS (B=30, BAMBU n=24, DYNAMATIC n=12 PER CONFIG). CONFIGURATIONS ARE GROUPED BY WHICH COMPONENTS ARE ACTIVE.

Configuration	Bambu		Dynamatic	
	SR (%)	Wasted	SR (%)	Wasted
no-filter	12.2 ±9.8	26.3	50.8 ±16.1	14.8
SCF-only	17.8 ±6.5	24.7	50.8 ±16.1	14.8
SCF + RPE	57.8 ±1.6	12.7	50.5 ±16.4	14.8
SCF + OFRS	84.5 ±3.2	4.7	89.5 ±6.5	3.2
SCF + DFRL (full)	**84.5** ±3.2	**4.7**	**89.5** ±6.5	**3.2**
DFRL-only	80.0 ±7.4	6.0	89.5 ±6.5	3.2
SCF + RPE-as-reorder	84.5 ±3.2	4.7	89.5 ±6.5	3.2
SCF + OFRS-as-skip (unsafe)	28.9 ±41.8	2.0	24.2 ±43.7	1.0

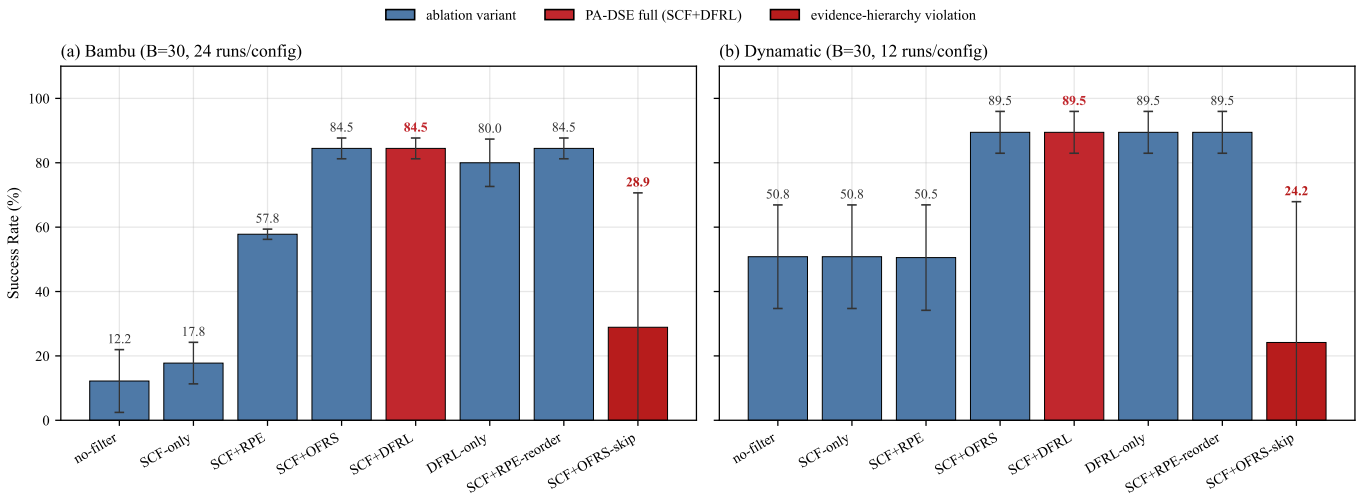

Fig. 8. Ablation study visualization (B=30). PA-DSE full configuration (SCF+DFRL) highlighted in red. The rightmost bar (SCF+OFRS-skip, dark red) violates the evidence hierarchy by granting OFRS skip authority based on weak per-dimension evidence, producing order-of-magnitude larger variance and mean SR collapse on both tools.

On some runs OFRS's first-order marginal statistics happen to align with true failure causes and the run succeeds; on others, OFRS prunes feasible configurations based on noisy single-dimension correlations and the run collapses. This is the strongest single piece of evidence for our design principle: *OFRS's weak evidence cannot justify the strong action of permanent exclusion*. When we allow it anyway, the consequences are severe.

f) Summary.: The ablation confirms three tenets of the evidence hierarchy: (i) SCF's permanent block is safe only because it uses provable evidence; (ii) OFRS's reordering is safe despite weak evidence because the action is weak (reorderable); (iii) crossing these—allowing OFRS to skip—breaks the hierarchy and performance collapses.

H. Overhead

Table V reports the per-iteration algorithmic overhead of each PA-DSE layer, compared to the corresponding synthesis cost. On Bambu, total PA-DSE overhead is 5.3 ms per iteration, versus $\approx$ 2504 ms of synthesis: a ratio of 1 : 474. On Dynamatic the ratio is 1 : 2908, reflecting the higher synthesis cost of MILP-based buffer placement. By either measure, algorithmic overhead is negligible.

OFRS dominates the algorithmic overhead (95% on Bambu, 92% on Dynamatic) because it performs a full-queue sort every iteration. SCF is essentially free (rules fire once at initialization), and RPE's overhead is concentrated in a small number of signature-extraction calls per run. For design spaces substantially larger than ours, the OFRS sort could be replaced by an incremental priority queue with $O(\log n)$ update—we have not needed this optimization for the scales evaluated here.

TABLE V
PER-ITERATION OVERHEAD DECOMPOSITION (MS). PA-DSE'S ALGORITHMIC COST IS $< 0.22\%$ OF SYNTHESIS COST ON BAMBU AND $< 0.04\%$ ON DYNAMATIC.

Tool	SCF	RPE	OFRS	Algo. total	Synthesis
Bambu	0.086	0.169	5.030	5.285	2503.7
Dynamatic	0.013	0.159	1.975	2.147	6244.6

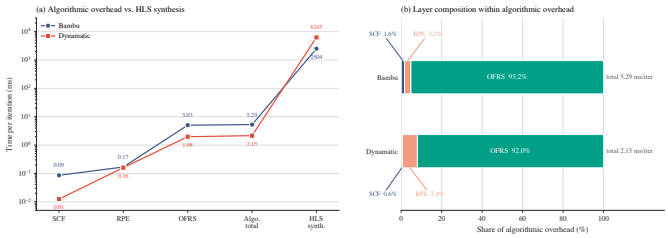

Fig. 9. Overhead analysis. (a) Per-iteration algorithmic cost is two-to-three orders of magnitude below synthesis cost on both tools. (b) Within the algorithmic overhead, OFRS dominates (it performs a queue sort each iteration); SCF and RPE are near-zero.

VI. DISCUSSION AND LIMITATIONS

A. When PA-DSE Helps, and When It Does Not

PA-DSE is most effective in the regime our evaluation targets: small-to-moderate exploration budgets on design spaces where a non-trivial fraction of configurations are infeasible. Outside this regime its advantages diminish, and we state the limits explicitly.

Very small budgets ($B < n_{\min}$). OFRS requires $n_{\min}$=5 observations per dimension before its relevance weights stabilize. Below this threshold, OFRS falls back to a passive mode that preserves the SCF-established prior (active configurations first, suppressed last). In this regime PA-DSE behaves like Filtered Random with a mild heuristic ordering bias. At B=10 we would expect the gap to RF to narrow.

Large budgets ($B \gg |\mathcal{C}_{\mathbf{act}}|$). At budgets approaching the full active-set size, every method converges to the same coverage. The pertinent question becomes not SR but QoR quality, where learned surrogates with offline training can leverage more data. PA-DSE is not designed for this regime.

Tools without documented incompatibility rules. On Dynamatic, SCF reduces to a no-op and PA-DSE's lead over the best baseline narrows from $+6.5$ pp to $+1.9$ pp. DFRL still helps (TTFF drops nearly $2\times$), but the margin over learned surrogates is smaller. For tools whose full failure distribution is driven by runtime-detected issues (e.g., timing closure, resource overflow), DFRL's online learning is the only applicable lever.

B. Quality–Reliability Tradeoff

Figure 7 shows that PA-DSE's pairwise QoR coverage is non-negative against every baseline: reliability and quality are not trading against each other at the per-method level. A subtler form of the tradeoff appears when comparing PA-DSE to the *union* of all six baselines: on Dynamatic, the collective baseline exploration finds a small number of Pareto-dominated interior points that PA-DSE misses because OFRS concentrates the budget on low-risk regions. If a user's goal is maximum QoR *diversity* beyond the Pareto front rather than maximum feasible-design count, tuning p_{probe} upward (accepting more "questionable" configurations) would recover diversity at the cost of some SR. A principled knob for this tradeoff is future work.

C. Decision Traceability

A consequence of PA-DSE's design is that every skip decision is attributable to either (a) a named static rule or (b) a named RPE signature with a known support count and confidence. At the end of a run, one can inspect `signatures_learned` and the associated `conditions` columns to see exactly which parameter combinations the algorithm treated as failure-prone. We call this *decision traceability* rather than *interpretability* to be precise: the framework exposes *why the policy took an action*, but does not claim a causal explanation for the underlying tool's failure—a signature captures a statistical conjunction, not a root cause. This is still a pragmatic advantage over surrogate-based methods, where a confidently-predicted infeasibility is not attributable to a named piece of state at all.

D. Generalization Across Runs

PA-DSE's DFRL state does not persist across runs. Each invocation starts with empty RPE signatures and empty OFRS statistics, re-learning within the budget. This is a deliberate choice: failure signatures are strongly benchmark-dependent (e.g., a pipeline accumulation conflict appears only in kernels with accumulation loops), and persisting them across benchmarks risks false skips. Cross-run persistence for the *same* benchmark is a natural extension—hashing the source AST and caching previously-learned signatures would yield a warm-start variant. We have not implemented this.

E. Threats to Validity

Tool versions. Our evaluation uses Bambu 0.9.8 and Dynamatic 2.0 with Gurobi 12.0. Different tool versions may expose different failure modes; the RPE signatures learned for Bambu 0.9.8 are not guaranteed to transfer to future releases.

Benchmark suite. Our 12 benchmarks are small-to-medium in size ($<$ 1000 lines of C each). For kernel-style benchmarks this is representative; for full-application HLS, longer compile times would shift the overhead ratio further in PA-DSE's favor, but the feasibility landscape may differ qualitatively.

Permutation-based repetition. For PA-DSE we report 10 queue permutations rather than 10 seeds. The two are mechanically equivalent for a deterministic algorithm (permutation

fixes the initial queue order, a seed fixes internal randomness in a stochastic algorithm), but readers should note this is not a statistical sample of PA-DSE's stochasticity per se—PA-DSE is deterministic conditional on the queue order. The $p_{\text{probe}}{=}0.05$ introduces residual stochasticity which is reflected in the observed $\sigma{=}1.1$ on Bambu.

No direct comparison against recent surrogate methods. We did not retrain recent GNN-based feasibility predictors on our configuration spaces. The RF baseline is our proxy for this family. A direct comparison against, e.g., a freshly-trained AutoScaleDSE or IronMan predictor would strengthen the claim that PA-DSE is competitive at low budget without offline training.

VII. Conclusion

We presented PA-DSE, a feasibility-aware HLS design space exploration policy built around the principle that action strength must not exceed evidence strength. The policy has two layers: a static constraint filter (SCF) that permanently removes configurations matching tool-documented incompatibility rules, and a dynamic failure risk learner (DFRL) that accumulates evidence during a single exploration run and reorders the queue accordingly. DFRL has two sub-components—recurrent pattern extraction (RPE) for signature-based hard skipping and online failure risk scoring (OFRS) for continuous queue reordering—each restricted to actions its evidence can justify.

On eight Bambu benchmarks at $B{=}60$, PA-DSE achieves 92.5% success rate with the lowest variance of any non-trivial method, outperforming a random-forest baseline by $+6.5$ pp and reducing wasted synthesis calls by $\mathbf{2}\times$. On four Dynamatic benchmarks at $B{=}30$—a tool with no applicable static rules, where SCF reduces to a no-op—PA-DSE still reaches 92.0% success rate and halves the time-to-first-feasible compared to the strongest baseline. An 8-way ablation confirms that each component contributes in the way its evidence hierarchy position predicts: OFRS's weak-evidence reordering is the largest per-component contributor to SR; SCF's strong-evidence permanent block is a safety net on tools that expose one; and allowing OFRS to hard-skip (breaking the hierarchy) causes variance to explode by an order of magnitude. The total algorithmic overhead is 5.3 ms per iteration on Bambu and 2.1 ms on Dynamatic, negligible compared to the 2.5 s and 6.2 s per-call synthesis costs.

The framework is training-free, single-run, and *decision-traceable*: every skip is attributable to a named rule or signature with a known support count. We do not claim causal explanation of the tool's underlying behaviour. We view the principal contribution as a design pattern—strictly couple action strength to evidence strength—rather than a specific set of rules or scoring functions. Extensions we consider promising include cross-run signature caching for repeated benchmark exploration, higher-order interaction-aware risk scoring for spaces where single-dimension statistics saturate, and integration with a probe-rate controller that actively modulates the quality–reliability tradeoff.

Code, raw experimental data, and a reproduction script for every table and figure in this paper are released at https://github.com/jane09250410/hls-dse.

References

[1] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, "AutoScaleDSE: A scalable design space exploration engine for high-level synthesis," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 3, pp. 1–30, 2023, the random-forest classifier baseline in our experiments is modelled after AutoScaleDSE's scalable surrogate.

[2] L. Ferretti, G. Ansaloni, and L. Pozzi, "Lattice-traversing design space exploration for high-level synthesis," in *Proceedings of the 36th IEEE International Conference on Computer Design (ICCD)*. IEEE, 2018, pp. 210–217, representative Bayesian-optimization-based DSE; our GP-BO baseline follows this family.

[3] C. Pilato and F. Ferrandi, "Bambu: A modular framework for the high level synthesis of memory-intensive applications," in *Proceedings of the 23rd International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2013, pp. 1–4.

[4] L. Josipović, R. Ghosal, and P. Ienne, "Dynamically scheduled high-level synthesis," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, 2018, pp. 127–136.

[5] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems 25 (NeurIPS)*, 2012, pp. 2951–2959.

[6] L. Ferretti, G. Ansaloni, and L. Pozzi, "Cluster-based heuristic for high level synthesis design space exploration," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 1, pp. 35–43, 2021.

[7] H.-Y. Liu and L. P. Carloni, "On learning-based methods for design-space exploration with high-level synthesis," in *Proceedings of the 50th Annual Design Automation Conference (DAC)*. ACM, 2013, pp. 1–7.

[8] N. Wu, Y. Xie, and C. Hao, "IronMan: GNN-assisted design space exploration in high-level synthesis via reinforcement learning," in *Proceedings of the 2021 Great Lakes Symposium on VLSI (GLSVLSI)*. ACM, 2021, pp. 39–44.

[9] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, "Automated accelerator optimization aided by graph neural networks," *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pp. 55–60, 2022, representative of the learned-classifier family used as our RF baseline.

[10] H. Kuang, X. Cao, J. Li, and L. Wang, "HGBO-DSE: Hierarchical GNN and Bayesian optimization based HLS design space exploration," in *Proceedings of the 2023 International Conference on Field Programmable Technology (ICFPT)*. Yokohama, Japan: IEEE, 2023, pp. 106–114, combines a hierarchical GNN QoR predictor with a tree-structured modeler that removes invalid configurations before Bayesian optimization—a close cousin of our SCF+DFRL layering.

[11] T. Koike-Akino, P. Wang, K. Kim, M. Pajovic, P. Millar, D. S. Yonge-Mallo, and K. Parsons, "AutoHLS: Learning to accelerate design space exploration for HLS designs," *arXiv preprint arXiv:2403.10686*, 2024, integrates a DNN feasibility predictor with Bayesian optimization. Related to our GP-BO baseline.

[12] E. Ustun, C. Deng, D. Pal, Z. Li, and Z. Zhang, "Accurate operation delay prediction for FPGA HLS using graph neural networks," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2020, pp. 1–9.

[13] E. Murphy and L. Josipović, "Balor: HLS source code evaluator based on custom graphs and hierarchical GNNs," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. Newark, NJ, USA: IEEE, 2024, pp. 1–9, gNN-based QoR estimator targeting the Dynamatic HLS flow.